

Many-to-Many Relationships

- Due No Due Date
- Points 1
- Submitting a website url

FORK

<https://github.com/learn-co-curriculum/python-p3-sqlalchemy-many-to-many/fork>  <https://github.com/learn-co-curriculum/python-p3-sqlalchemy-many-to-many>  <https://github.com/learn-co-curriculum/python-p3-sqlalchemy-many-to-many/issues/new>

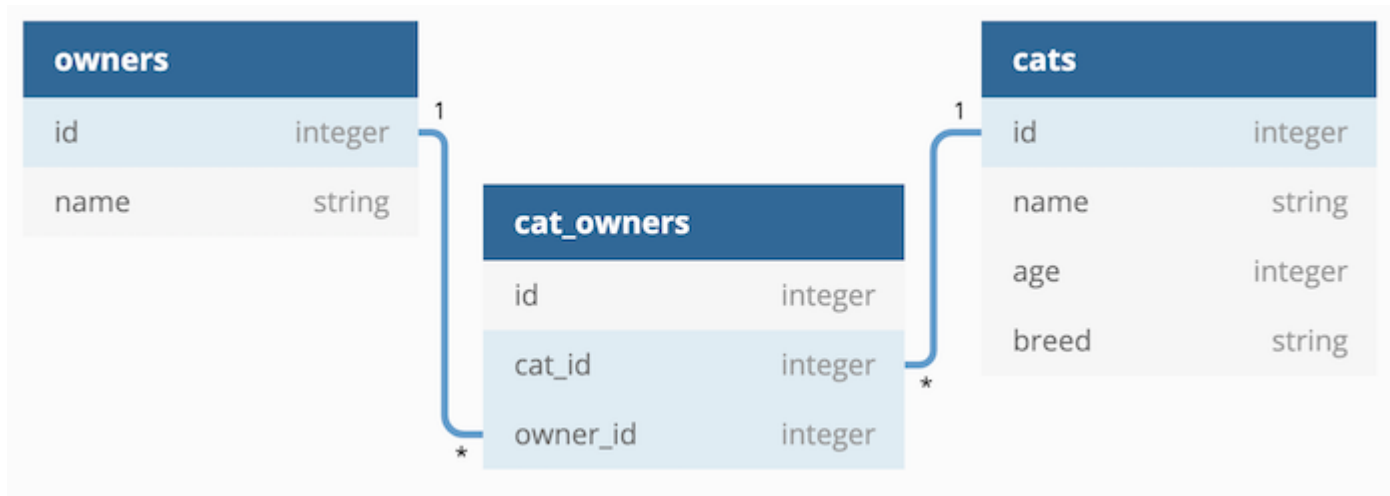
Learning Goals

- Use SQLAlchemy to join tables with one-to-one, one-to-many, and many-to-many relationships.

Introduction

In the previous lesson, we saw how to create a **one-to-many** association between two models using SQLAlchemy by following certain naming conventions, use of the `relationship()` and `backref()` methods, and using the right foreign key on our tables when generating the migrations.

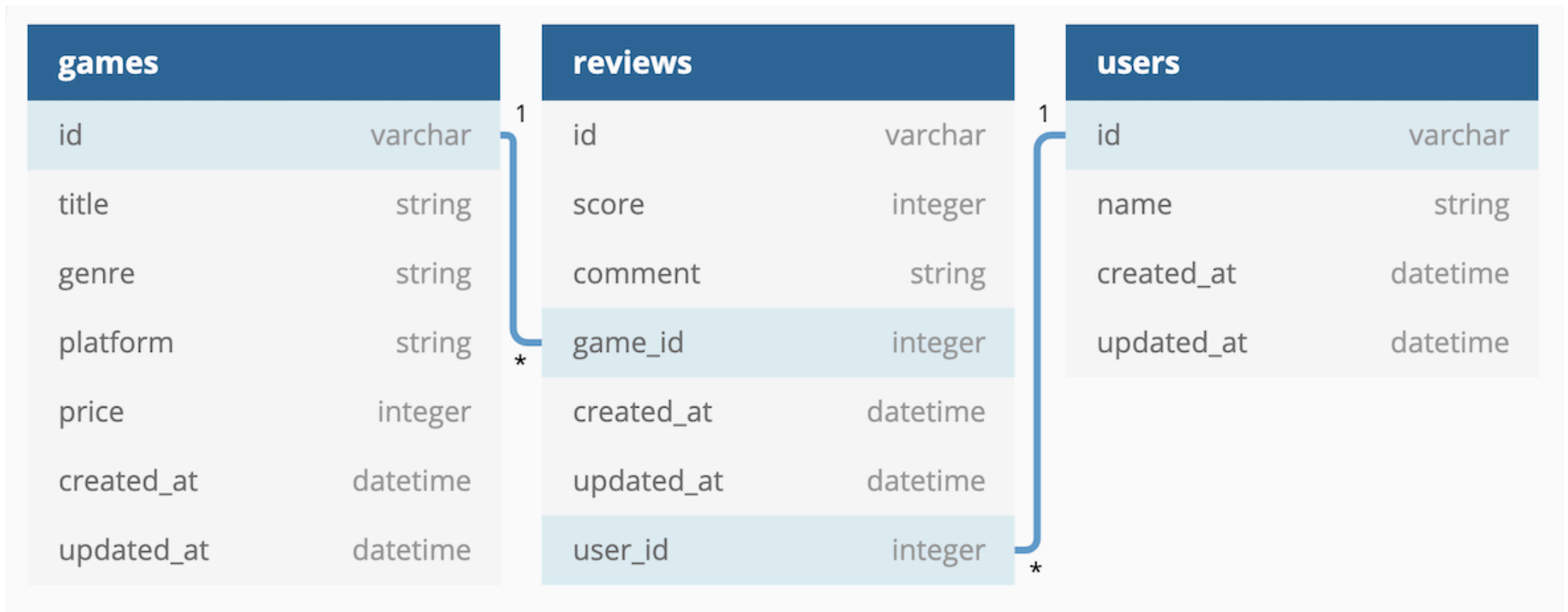
In the SQL section, we learned about one other kind of relationship: the **many-to-many**, also known as the **has many through**, relationship. For instance, in a domain where a **cat** has many **owners** and an **owner** can also have many **cats**, we needed to create another table to join between those two tables:



In this lesson, we'll learn how to create a **many-to-many** relationship in SQLAlchemy. We'll continue working on our games and reviews domain, but this time we'll add a third model into the mix: a users model. We'll be setting up these relationships:

- A game **has many** reviews.
- A game **has many** users, **through** reviews.
- A review **belongs to** a game.
- A review **belongs to** a user.
- A user **has many** reviews.
- A user **has many** games, **through** reviews.

Once we're done setting up the database tables, here's what the ERD will look like:



To get started, run `pipenv install && pipenv shell`, then follow along with the code.

Creating a User Model

Right now, we've got code for the `Game` model (and the `games` table), along with the code for the `Review` model (and the `reviews` table) from the previous lesson.

To start, let's add the code we'll need for the `User` model as well. Let's create the `users` table with a `name` column and timestamps:

```
# models.py
```

```
class User(Base):
```

```
__tablename__ = 'users'

id = Column(Integer(), primary_key=True)
name = Column(String())
created_at = Column(DateTime(), server_default=func.now())
updated_at = Column(DateTime(), onupdate=func.now())

# don't forget your __repr__()!
def __repr__(self):
    return f'User(id={self.id}, ' + \
           f'name={self.name})'
```

Note that for our models' timestamps, we are using some new arguments and values:

- `server_default` tells the database schema to set a value from the database itself. Since the database is kept in one central location, assigning it the work of creating default values means that we don't have to worry about the quality of our developers' or users' computers.
- `onupdate` means exactly what it says: when the record is updated, the column value is set.
- We saw `func` briefly in the previous module; it allows us to use SQL operations instead of their Python counterparts. This benefits us for the same reasons as `server_default`. `func.now()` is equivalent to the current time.

We'll also need to modify the `reviews` table and add a foreign key to refer to our `users` table. Remember, each review now **belongs to** a specific user. Any time we create a **belongs to** relationship, we need a foreign key to establish this relationship:

```
# models.py
```

```
class User(Base):

    # tablename, columns

    reviews = relationship('Review', backref=backref('user'))
```

```
# __repr__()
```

Let's also edit the `Review` model to add our new foreign key:

```
# models.py
```

```
class Review(Base):
```

```
    # tablename, columns
```

```
    user_id = Column(Integer(), ForeignKey('users.id'))
```

```
    # __repr__()
```

Now run `alembic revision --autogenerate -m'Add User model'` from the `lib` directory to make our migration. If all goes well, run `alembic upgrade head` to push your migrations to the database.

```
$ alembic upgrade head
```

```
# => INFO [alembic.runtime.migration] Context impl SQLiteImpl.
```

```
# => INFO [alembic.runtime.migration] Will assume non-transactional DDL.
```

```
# => INFO [alembic.runtime.migration] Running upgrade -> 9e396fc70825, Add User model
```

Run the first seed file as well to populate the `games` and `reviews` tables:

```
$ python seed.py
```

Creating a Many-to-Many Relationship

There are several ways to approach a many-to-many relationship in SQLAlchemy. All require some sort of intermediary between the two models. We could do this using the `Review` model, but this isn't the best choice. What if we wanted to add functionality in the future where users could log their games without adding reviews? In any case, it's unlikely that reviews will be the only things tying users and games to one another.

Many-to-many relationships in SQLAlchemy use intermediaries called **association tables**. These are tables that exist only to join two related tables together. This might sound like a waste at first, but keeping the foreign keys, relationships, and `backref` s confined to this one table allows us to freely make changes to our related tables later on without worrying too much about how they might affect each other.

There are two approaches to building these associations: association objects, which are most similar to the models we've built so far, and the more common approach, `Table` objects.

Many-to-Many with an Association Object

An association object is really just another model, so we can create a `GameUser` model using our `Base` object and simply build relationships in either direction, as seen below:

```
# example only
```

```
class GameUser(Base):
    __tablename__ = "game_users"

    id = Column(Integer(), primary_key=True)
    game_id = Column(ForeignKey('games.id'))
    user_id = Column(ForeignKey('users.id'))

    game = relationship('Game', back_populates='game_users')
    user = relationship('User', back_populates='game_users')

    def __repr__(self):
        return f'GameUser(game_id={self.game_id}, ' + \
            f'user_id={self.user_id})'
```

An association object can use either its own primary key or a combination of the two joined tables' primary keys as a unique identifier. Here, we use the simpler strategy and create an `id` column.

Next, we use the `relationship()` method to connect to both the `Game` and `User` models. Here, we opt for the `back_populates` argument in place of `backref`. This is because when we're building out a many-to-many relationship, *many* things can go wrong. In this case, it's difficult to use `backref` to populate each model with all the fields it needs without accidentally doing it twice. `back_populates` needs to be set up on both sides, but in cases like these, it's worth the (minimal) extra work.

Finally- don't forget your `__repr__` !

While this approach works, it's a bit wordy for a table that only exists to connect two others. The preferred approach to define many-to-many relationships using SQLAlchemy is with `Table` objects.

Many-to-Many with `Table` Objects

`Table` objects are instances of the `sqlalchemy.Table` class. They function more or less the same as data models, with the exception of being a little more compact. This syntax visually de-emphasizes association tables in your models and is the preferred approach in SQLAlchemy.

Let's build the same association table as above with our new syntax:

```
# models.py

game_user = Table(
    'game_users',
    Base.metadata,
    Column('game_id', ForeignKey('games.id'), primary_key=True),
    Column('user_id', ForeignKey('users.id'), primary_key=True),
    extend_existing=True,
)

class Game(Base):

    # tablename, columns
```

```
users = relationship('User', secondary=game_user, back_populates='games')

# __repr__()
```

```
class User(Base):

    # tablename, columns

    games = relationship('Game', secondary=game_user, back_populates='users')

    # __repr__()
```

Because we are creating an object that is being used in subsequent code, association tables made from the `Table` class must be defined above other data models. Otherwise, they wouldn't exist when they were referenced!

A few more notes on this approach:

- The `Game` and `User` models each require a relationship with the other.
- The `secondary` argument refers to the intermediary table in a many-to-many relationship.
- The `back_populates` operates similarly to `backref`, with the exception that it must be used on both sides of a relationship. Because many-to-many relationships are symmetrical, use of `back_populates` in both models is the best way to leave readable code behind for other developers.

Run `alembic revision --autogenerate -m'Add game_user Association Table'`, then `alembic upgrade head`. You can use the script in `app/seed_2.py` to generate new data and interact with your database through the Python shell. To create relationships between `Game` records and `User` records, run the second seed file with `python seed_2.py`.

This will add a `Game` record to a `User` record's `games` if the user has logged a review for the game. When the change is committed, SQLAlchemy also builds the relationship in reverse, adding the `User` record to the `Game` record's `users` !

Bonus: Association Object Models

We all know that users can have games without reviewing them. That being said, our application might not allow users to claim ownership of a game without posting a review! If this is the case, we can consider skipping a join table entirely and use `Review` to join `User` and `Game`. This syntax is a bit more complicated, but you might find it useful in certain situations (perhaps like the *Phase 3 Code Challenge?*)

An association object functioning as a traditional data model looks like a combination of the two, with some key differences:

NOTE: Remember to delete your `Table` object if you intend to try this out!

example only

```
class Game(Base):
    __tablename__ = 'games'

    id = Column(Integer(), primary_key=True)
    title = Column(String())
    genre = Column(String())
    platform = Column(String())
    price = Column(Integer())
    created_at = Column(DateTime(), server_default=func.now())
    updated_at = Column(DateTime(), onupdate=func.now())

    reviews = relationship('Review', back_populates='game')
    users = association_proxy('reviews', 'user',
                             creator=lambda us: Review(user=us))

    def __repr__(self):

        return f'Game(id={self.id}, ' + \
            f'title={self.title}, ' + \
            f'platform={self.platform})'

class User(Base):
    __tablename__ = 'users'
```

```

id = Column(Integer(), primary_key=True)
name = Column(String())
created_at = Column(DateTime(), server_default=func.now())
updated_at = Column(DateTime(), onupdate=func.now())

reviews = relationship('Review', back_populates='user')
games = association_proxy('reviews', 'game',
    creator=lambda gm: Review(game=gm))

def __repr__(self):

    return f'User(id={self.id}, ' + \
        f'name={self.name})'

```

See the most important piece below:

```

class Review(Base):
    __tablename__ = 'reviews'

    id = Column(Integer(), primary_key=True)

    score = Column(Integer())
    comment = Column(String())
    created_at = Column(DateTime(), server_default=func.now())
    updated_at = Column(DateTime(), onupdate=func.now())

    game_id = Column(Integer(), ForeignKey('games.id'))
    user_id = Column(Integer(), ForeignKey('users.id'))

    game = relationship('Game', back_populates='reviews')
    user = relationship('User', back_populates='reviews')

```

```
def __repr__(self):  
  
    return f'Review(id={self.id}, ' + \  
           f'score={self.score}, ' + \  
           f'game_id={self.game_id})'
```

The `Review` model has gotten rather big here, but it's doing more than enough to justify it:

- `game_id` creates a relationship between reviews and games.
- `game` creates a game object that belongs to the review.
- `user_id` creates a relationship between reviews and users.
- `user` creates a user object that belongs to the review.

Because of those object relationships, the `relationship()` call in the `Game` model is able to skip over the `Review` model and directly link games and users.

Notice in the `Game` and `User` models that we have added an `association_proxy` (imported from `sqlalchemy.ext.associationproxy`) to refer to the many-to-many related table. This states that there is an association through the `reviews` table's `game` or `user` column. The `creator` argument takes a function (an anonymous `lambda` function in this case) which accepts a game or user and returns a review for that game or user. This review has, in a sense, created the relationship between the game and user.

As we mentioned earlier, this syntax is a bit complicated, and `Table` objects are still generally preferred. It is rare that the only thing joining two tables would be another concrete table like `reviews`. Still, these cases exist and some developers prefer to minimize the number of tables in their databases either way. Association Object Models are always good to have in your back pocket.

A testing suite is available in this lesson for you to check your syntax in building a many-to-many relationship between games and users. Run `pytest -x` from the `lib/` directory to see if your models are working as expected- and don't forget to use Alembic to create your database first!

Conclusion

The power of SQLAlchemy all boils down to understanding database relationships and making use of the correct classes and methods. By leveraging "convention over configuration", we're able to quickly set up complex associations between multiple models with just a few lines of code.

The **one-to-many** and **many-to-many** relationships are the most common when working with relational databases. You can apply the same concepts and code we used in this lesson to any number of different domains, for example:

```
Driver -< Ride >- Passenger
Doctor -< Appointment >- Patient
Actor -< Character >- Movie
```

The code required to set up these relationships would look very similar to the code we wrote in this lesson.

By understanding the conventions SQLAlchemy expects you to follow, and how the underlying database relationships work, you have the ability to model all kinds of complex, real-world concepts in your code!

Solution Code

```
# many_to_many/models.py

from sqlalchemy import create_engine, func
from sqlalchemy import ForeignKey, Table, Column, Integer, String, DateTime
from sqlalchemy.orm import relationship, backref
from sqlalchemy.ext.declarative import declarative_base

engine = create_engine('sqlite:///many_to_many.db')

Base = declarative_base()

game_user = Table(
    'game_users',
    Base.metadata,
```

```
Column('game_id', ForeignKey('games.id'), primary_key=True),
Column('user_id', ForeignKey('users.id'), primary_key=True),
extend_existing=True,
)

class Game(Base):
    __tablename__ = 'games'

    id = Column(Integer(), primary_key=True)
    title = Column(String())
    genre = Column(String())
    platform = Column(String())
    price = Column(Integer())
    created_at = Column(DateTime(), server_default=func.now())
    updated_at = Column(DateTime(), onupdate=func.now())

    users = relationship('User', secondary=game_user, back_populates='games')
    reviews = relationship('Review', backref=backref('game'))

    def __repr__(self):
        return f'Game(id={self.id}, ' + \
            f'title={self.title}, ' + \
            f'platform={self.platform})'

class User(Base):
    __tablename__ = 'users'

    id = Column(Integer(), primary_key=True)
    name = Column(String())
    created_at = Column(DateTime(), server_default=func.now())
    updated_at = Column(DateTime(), onupdate=func.now())

    games = relationship('Game', secondary=game_user, back_populates='users')
```

```
reviews = relationship('Review', backref=backref('user'))

def __repr__(self):
    return f'User(id={self.id}, ' + \
           f'name={self.name})'

class Review(Base):
    __tablename__ = 'reviews'

    id = Column(Integer(), primary_key=True)

    score = Column(Integer())
    comment = Column(String())
    created_at = Column(DateTime(), server_default=func.now())
    updated_at = Column(DateTime(), onupdate=func.now())

    game_id = Column(Integer(), ForeignKey('games.id'))
    user_id = Column(Integer(), ForeignKey('users.id'))

    def __repr__(self):
        return f'Review(id={self.id}, ' + \
               f'score={self.score}, ' + \
               f'game_id={self.game_id})'
```

Resources

- [Python 3.8.13 Documentation](https://docs.python.org/3/) ➞ (https://docs.python.org/3/)
- [SQLAlchemy ORM Documentation](https://docs.sqlalchemy.org/en/14/orm/) ➞ (https://docs.sqlalchemy.org/en/14/orm/)
- [Alembic 1.8.1 Documentation](https://alembic.sqlalchemy.org/en/latest/) ➞ (https://alembic.sqlalchemy.org/en/latest/)

- [Basic Relationship Patterns - SQLAlchemy](https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html)  (https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html)